仕様どおりに実装すれば安全か — イーサリアムのコンセンサス仕様に明示されない安全性の前提条件（予備調査）—

村上 大岳[1,a)]　神波 真聖[2,b)]　柳浜 万里[2,c)]　釘宮 由成[3,d)]

概要：イーサリアムでは，複数のクライアントが共通の仕様書に従って合意形成を行う．しかし仕様準拠だけでは，確定したブロックが覆らないという安全性を保証するための前提条件まで確認できない．本研究は，ファイナリティ機構 Casper FFG の安全性を Lean 4 で形式化した証明から，定理が依存する前提条件を機械的に抽出し，仕様書の自然言語記述に安全性要件として明示されているかを照合した．延べ 57 件の前提条件のうち 48 件（84.2%）は，キーワード照合器と 6 種類の大規模言語モデルによる 7 つの自動判定すべてで明示されていないと判定された．本研究は，形式証明から安全性の前提条件を監査項目へ機械的に変換し，仕様準拠監査と安全性証明の差分を測定する枠組みを示す．抽出した一覧は，Ethereum クライアントの追加監査項目候補として提供する．

キーワード：形式検証，Lean 4，コンセンサスプロトコル，LLM 監査，Ethereum

Does Conformance Imply Safety? Safety Preconditions Not Stated in Ethereum's Consensus Specification (Preliminary Study)

Hirotake Murakami[1,a)]　Masato Kamba[2,b)]　Banri Yanahama[2,c)]　Yoshinari Kugimiya[3,d)]

Abstract: Ethereum is maintained by multiple clients that follow a common consensus specification. Conformance alone, however, does not show that the preconditions required for finality—the irreversibility of a finalized block—are satisfied. We mechanically extract such preconditions from a Lean 4 formalization of the safety of Casper FFG, Ethereum's finality mechanism, and check whether they appear as safety requirements in the natural-language specification. Of 57 proof-derived preconditions, 48 (84.2%) were unanimously judged not explicit by seven automated annotators: a keyword matcher and six LLMs. This study extracts safety preconditions that conformance review alone may overlook, quantifies their range, and offers the extracted list as candidate audit items for Ethereum clients.

Keywords: formal verification, Lean 4, consensus protocol, LLM auditing, Ethereum

[1] 愛知県立愛知総合工科高等学校
Aichi Prefectural Aichi High School of Technology and Engineering
[2] Nyx Foundation
Nyx Foundation
[3] Independent Researcher
Independent Researcher
[a)] 2009hirotake@gmail.com
[b)] kingmasatojames@gmail.com
[c)] banri.yanahama@gmail.com
[d)] 0xfacad3@gmail.com

1. はじめに

イーサリアム（Ethereum）は，世界中に分散した多数のノードが同じ取引履歴を共有するブロックチェーンである．その運用上の特徴の一つは，単一の公式ソフトウェアが存在しないことである．どのような入力を受け取ったときに内部状態をどう更新するかを定めた共通の仕様書が公開されており，複数の開発チームがそれぞれ独立に実装（クラ

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

イアントと呼ばれる）を作り，同時に稼働させている．実装が多様であることは，特定の実装のバグがネットワーク全体を巻き込む事態を避けるうえで利点となる一方，異なる実装が同じ状態に到達する根拠は，共通の仕様書とその解釈だけに置かれる．そのためクライアントの監査は，仕様書に書かれた処理を正しく実装しているかという仕様準拠の確認が中心になる．しかし，仕様どおりに実装されていることと，攻撃や障害の下でも安全性が保たれることは同じではない．

本研究が対象とする安全性は，ファイナリティ，すなわち一度確定（finalize）したブロックが後から覆らないという性質である．ブロックチェーンでは，取引が確定したという判断のうえで取引所の出金や他システムへの連携が行われるため，確定済みのブロックが覆ると，取引の巻き戻りや二重支払いが生じ，決済の不可逆性そのものが崩れる．したがってファイナリティは，可用性の低下にとどまらず資産の損失に直結する安全性要件である．

イーサリアムでこのファイナリティを支えるのが Casper FFG と呼ばれる投票の仕組みである．資産を預託した参加者は検証者（validator）と呼ばれ，どのブロックを正しいと考えるかを投票する．ブロックの確定は，全体の 3 分の 2 以上の重みを持つ投票の集合（本稿ではこれをクォーラム（quorum）と呼ぶ）に支えられており，確定に至る条件の詳細は 2 章で述べる．Casper FFG について証明されている accountable safety（帰責可能な安全性）とは，相反する二つのブロックが同時に確定するという事態が仮に起きたならば，矛盾する投票を行った検証者を必ず特定でき，預託資産の没収（スラッシング）の対象にできる，という性質である．この性質は数学的に証明されているが，証明はいくつかの前提条件のもとで結論が成り立つという含意の形をとる．たとえば，相反する二つのブロックをそれぞれ支える投票集合が必ず重なること，その重なりに属する検証者が矛盾する投票を行ったものとして実際に検出され処罰されること，といった条件である．すなわち定理の結論は，これらの前提を満たす範囲に適用されるものであり，実装や運用がその範囲に収まっているかどうかは，証明とは別に確認する必要がある．

ここに，仕様準拠監査だけでは扱えない範囲が生じる．仕様書は，各クライアントが同じ状態遷移を行うために必要な処理手順を記述するものであり，安全性証明がどの前提条件に依拠しているかを安全性要件として書き下すことは目的としていない．監査項目を仕様書の記述から列挙する限り，仕様書に現れない前提条件は監査の入力にも現れにくく，結果として確認されないまま残りやすい．これは実装に脆弱性があることを直接示すものではない．仕様書に基づく監査が，安全性証明の観点からどこまでを対象にできているか，という監査範囲の問題である．

近年，大規模言語モデル（LLM）に仕様書を読ませてセキュリティプロパティを生成し，そのプロパティに照らして実装を検査する監査手法（Specification-Anchored Auditing，以下 SPECA）が報告されている [1], [2]．この種の手法は仕様書を主たる入力とするため，前段落で述べた範囲の問題をそのまま引き継ぐことになる．本研究はこの課題に対し，安全性証明の側から監査項目を供給する経路を設ける．具体的には，定理証明支援系 Lean 4 で形式化された安全性証明を入力とし，各定理が依存する前提条件を機械的に抽出したうえで，それらが仕様書に安全性要件として明示されているかを 1 件ずつ照合し，仕様書と安全性証明の差分を測定する．

本研究の主たる貢献は，形式証明が要求する前提条件を仕様書ベースの準拠監査で扱える監査項目へ変換し，仕様書と安全性証明の差分を測定する監査設計の方法論を示したことである．Ethereum のコンセンサスは，この方法論を具体化するためのケーススタディとして選んだ対象であり，得られた項目一覧はその成果物である．以下に本研究の貢献を整理する．

(1) **形式証明の前提条件を監査項目へ変換する手順を示した**: 安全性証明の定理が仮説として置いている条件を機械的に列挙し，仕様書の自然言語記述と照合可能な単位に整形する変換を定義した．この手順は，形式証明と仕様書がともに存在するシステムであれば，同様の形での適用を検討できる．

(2) **仕様準拠監査と安全性証明の差分を実プロトコルで定量化した**: ケーススタディとして，Lean 4 で形式化された Casper FFG の安全性証明を対象に，17 定理から延べ 57 件の前提条件を抽出し，仕様書と 1 件ずつ照合した．このうち 48 件（84.2%）が，キーワード照合器 1 種と 6 種類の LLM による計 7 つの自動判定すべてで「安全性要件として明示されていない」と判定された．

(3) **形式証明と仕様準拠監査を接続する成果物を示した**: 抽出した前提条件を，Lean の証明を直接読まずに参照できる監査項目の一覧として付録 B に示す．この一覧は，Ethereum クライアントの監査において追加確認の候補となる項目であり，上記の変換手順が何を出力するのかを具体的に示すものでもある．

本事例は，仕様書と形式化された安全性証明の双方をもつ他の高信頼システムに対しても，両者の差分を監査項目として扱う設計の適用可能性を検討するための枠組みを与える．ただし本研究が扱うのは，形式証明から監査プロパティを生成する段階までであり，生成した項目一覧を用いた実際の脆弱性検出性能や監査の有効性は評価していない（6 章）．

© 2026 Information Processing Society of Japan

2. 背景

2.1 SPECA パイプライン

SPECA（Specification-Anchored Auditing）は，仕様書からセキュリティプロパティを生成し（フロントハーフ），クライアントコードを監査する（バックハーフ）パイプラインである [2]．フロントハーフは `01a`（収集）→ `01b`（抽出）→ `01e`（プロパティ生成），バックハーフは `02c`（コード位置特定）→ `03`→`04`→`05` からなる．

Phase `01e` のプロパティ生成には，入力の与え方が異なる複数の生成器（provider）を差し替えられる．従来の prompt provider は，仕様テキストをプロンプトとして LLM に与え，プロパティを自由生成させる．これに対し本稿が提案する lean provider は，Lean 4 の形式証明から前提条件を機械的に導出してプロパティ化する．この経路は前提条件の抽出に LLM を介さない（決定性は 3.1 節で述べる）．本稿の分析対象はフロントハーフの出力であるプロパティに限り，監査性能の評価はスコープ外である（6 章）．

2.2 Casper FFG

Casper FFG（Friendly Finality Gadget）は，Ethereum のコンセンサスプロトコルにおけるファイナリティガジェットである [8]．ここでいう安全性（accountable safety）とは，「一度 finalize されたブロックは二度と覆らない」というファイナリティの保証に加えて，**もしそれが覆るなら違反した検証者を必ず特定してスラッシュできる**（帰責可能である）という形の安全性を指す．悪意ある検証者を全体の 1/3 未満と仮定して成立する通常の安全性と異なり，accountable safety はその仮定を置かず，違反が起きた場合に一定割合が必ずスラッシュ対象となることを結論する定理として証明される．この保証は quorum の交差性に依存する．各 finalization を支える 2/3 集合どうしは必ず全体の 1/3 以上で重なり（$2/3 + 2/3 - 1 = 1/3$．この算術は両集合が同一の検証者集合上で定義される場合のものであり，検証者集合が入れ替わる一般の場合は 2.3 節の定式化による），スラッシュ機構が正しく機能していれば，その重なりに含まれる二重投票者はスラッシュ対象となる（**図 1**）．この quorum 交差が健全でないと，相反する 2 つの finalization が同一高さで両立し，チェーンが分岐する（chain split）．chain split が起きると，一度確定したはずの取引が別の分岐では無かったことになり得るため，二重支払いや確定済み残高の消失といった，決済の不可逆性そのものが崩れる事態が起こり得る．

本研究では，`gasper-lean4` リポジトリにおいて Casper FFG を Lean 4 で形式証明した [10]．本稿が分析対象とするのは，形式化全体（25 定理・11 ファイル）のうち，

C（高さ h，確定）	C'（同高さ，相反）
↑ Q（2/3 集合）	↑ Q'（2/3 集合）

$Q \cap Q' \geq 1/3$　$(2/3 + 2/3 - 1)$

交差の投票者 = 二重投票 ⇒ スラッシュ対象

破れれば同時 finalize ⇒ chain split

図 1 quorum 交差と chain split：同一高さの相反する C と C' を支える 2/3 集合 Q, Q' は必ず 1/3 以上で重なり，その重なりは二重投票としてスラッシュ対象となる（¬`q_intersection_slashed` が対応）．この咎めが破れると C' も成立し chain split に至る．

Fig. 1 Quorum intersection and chain split.

後述の CHK-* 監査項目を導出した 17 定理である．この 17 定理は 8 ファイルにまたがり，リポジトリの分類では Core/Theories に 9 定理，Core/Lemmas に 4 定理，Executable に 4 定理が属する．うち安全性の核心をなすのは 5 定理である．17 定理はすべて実在し，`sorry` を含まない（証明項は Lean のカーネルが受理する）．ただしこれは，形式化したモデルが実プロトコルを忠実に表すことまでを保証するものではない．本稿で分析する 57 の前提条件（$n = 57$）はこの 17 定理に由来する．

再現性のため，本稿が参照する成果物のコミットを固定する：`gasper-lean4` = `7650709` [10]，consensus-specs = `2359a5e` [11]，speca-lean4-plugin の出力 = `20260723-gasper` [12]．前提条件は，plugin の出力をそのまま用いるのではなく，`gasper-lean4` の定理定義と全件照合したうえで確定した．pin した plugin 出力は 54 条件を含むが，このうち 1 定理（`slashable_bound`）については plugin が実ソースを解決できずテスト用の固定データ（fixture）にフォールバックしており，実ソースは 4 条件を持つ．本稿は実ソースの内容を採用し，全 57 条件を実物照合済みとする．抽出変換自体は決定的だが，入力が実ソースでなく fixture にフォールバックした場合には実在しない前提が混入しうる．この照合手順は実際にその混入を 1 件検出しており，実物照合を手続きに含めることの有効性を示す．

2.3 形式証明の骨子：accountable safety

前提抽出の対象である `gasper-lean4` の中核は accountable safety の定理である．本稿はこれを黒箱とせず，抽出する前提条件がどの定理の仮説に由来するかを示す．最上位の定理は，**finalization fork**（互いに祖先でない 2 ブロック b_1, b_2 がともに finalize された状態）が `q_intersection_slashed` を強制することを主張する：

$$\texttt{finalization_fork}(\sigma) \rightarrow \texttt{q_intersection_slashed}(\sigma).$$

ここで 2/3-クォーラム $\texttt{quorum_2}(q, b)$ は $q \subseteq V(b) \wedge f_{2/3}(\mathrm{wt}(V(b))) \leq \mathrm{wt}(q)$，すなわちブロック b の検証者集合 $V(b)$ の 2/3 以上の重みをもつ部分集合である．`q_intersection_slashed` は，2 つの 2/3-クォーラム q_L, q_R

© 2026 Information Processing Society of Japan

が存在してその交差に属する**すべて**の検証者がスラッシュされること（$\forall v \in q_L \cap q_R$, $\texttt{slashed}(\sigma, v)$）を表し，図 1 の網掛け領域に対応する．検証者集合をブロックごとに与える（$V(b_L) \neq V(b_R)$ を許す）ことで動的検証者集合にも一般化される．証明は k-ファイナライゼーション（1 に限らない）へ一般化した `k_safety'` をブロックの高さの三分岐で構成する: (i) 同一高さ → 二重投票（Casper の S1），(ii) surround 型 → surround 投票（S2），(iii) 高さ差に関する強帰納法で (i)(ii) に帰着．これは Casper FFG 定理 1 [8] ／ Gasper 定理 5.2 [9] を，k-ファイナライゼーションと動的検証者集合へ一般化したものである．

測定対象との接続． 本稿が抽出する前提条件は，これらの定理の**仮説**そのものである．主事例 CHK-GEN-09 の母定理 `no_k_finalized_justified_same_height` は，仮説に $\neg$`q_intersection_slashed` を含む形

$$\begin{aligned} &\texttt{k_finalized}(\sigma, b_f) \wedge \texttt{justified}(\sigma, b_j) \\ &\quad \wedge\ \neg\texttt{q_intersection_slashed}(\sigma) \wedge b_j \neq b_f \\ &\qquad \rightarrow\ b_{j,h} \neq b_{f,h} \end{aligned}$$

をもつ．この $\neg$`q_intersection_slashed` は，2 つのクォーラムの交差に属する全検証者がスラッシュされている状態が成立していないこと，すなわち帰責がまだ発火していないことを表す．`q_intersection_slashed` は最上位の定理では結論として現れ，補題 `no_k_finalized_justified_same_height` では仮説として現れる．本稿はこの仮説形を，実装が能動的に確立する条件ではなく，定理が依拠する脅威モデル上の仮定として計上している（分類上の限界は 6 章）．これは本稿が非 E と測る前提条件（**表 A·3** の #39）に一致する．すなわち安全性は「この仮定のもとで相反する finalize が同一高さに並ばない」という含意として証明されており，その仮定は仕様の散文に検査可能な要件として現れない．前提抽出（3 章）が拾うのは，まさにこの型シグネチャ上の仮説群である．

3. 提案手法

本手法は，形式証明を「1 回だけ書く」高コスト作業として隔離し，その成果を LLM を介さない機械変換で監査可能なプロパティ列に落とし，監査者が Lean を一切読まずに利用できるようにする 3 段階アーキテクチャである．

3.1 3 段階アーキテクチャ

本手法は，役割の異なる 3 段階に分業される（**図 2**）．分業の要点は，Lean を書ける・読める能力を要するのは Stage 1 の 1 回だけであり，下流の監査者はその能力を一切必要としない点にある．

Stage 1: 形式証明（証明の専門家が 1 回のみ）． 対象プロトコルの安全性定理を Lean 4 で形式証明する．本研究では Casper FFG の accountable safety を核として 11 ファイル・25 定理を形式化し，そのうち CHK-* を導出した 8 ファイル・17 定理を本稿の分析対象とする．この 17 定理の証明ステータスはすべて proved・`sorry` ゼロである．この作業は形式手法の専門家が担い，プロトコルごとに 1 回行えばよい．

Stage 1	**Stage 2**	**Stage 3**
形式証明	分解	監査
専門家・1 回	決定的変換	スコープ外
Lean の読み書きはここまで		

図 2 3 段階アーキテクチャと分業．Lean を読む／書く能力が要るのは Stage 1 のみ．Stage 2 は決定的な機械変換（LLM 不介在）で JSON を出力し，Stage 3（Phase `02c` 以降の監査）は本稿のスコープ外である．

Fig. 2 Three-stage architecture and division of labor.

Stage 2: must-establish 分解（機械変換，LLM 不介在）． Stage 1 の Lean 証明から，speca-lean4-plugin が定理の前提条件（hypotheses）を機械的に抽出し，Phase `01e` 形式のプロパティ JSON を生成する．この変換は Lean のメタプログラミングによる決定的な導出であり，抽出段で LLM による生成を行わないため，前提条件が捏造されることはない（入力の取り違えに対しては 2.2 節の実物照合を，抽出後の二値ラベリングの限界については 6 章を参照）．

Stage 3: クライアント監査（監査者は Lean を読まない）． Stage 2 が出力した JSON プロパティを入力として，Phase `02c` 以降が対象クライアントに対して監査を実行する．本稿は下流の監査を扱わない（6 章）．Lean の記述・読解は Stage 1 の専門家に隔離され，利用者が受け取るのは Stage 2 の JSON プロパティのみである．

利用者が Stage 2 から受け取る 1 プロパティの JSON 表現を，形式の一例として以下に示す（主事例 CHK-GEN-09）．利用者はこの JSON を Phase `02c` 以降に渡すだけでよく，`lean_must_establish` の各条件が「この定理が成立するために実装が満たすべき前提条件」を表す．

```
{
 "property_id":  "CHK-GEN-09",
 "text":  "Confirm quorum-intersection
  accountability, so that two conflicting
  k-finalizations at the same height
  cannot both hold.",
 "severity":  "CRITICAL",
 "lean_status":  "descends-from-proved",
 "lean_artifact":
  ".../AccountableSafety.lean#
   no_k_finalized_justified_same_height",
 "spec_reference":
  ".../phase0/beacon-chain.md#
   process_justification_and_finalization",
 "lean_must_establish":  [
  "¬ q_intersection_slashed ...  st",
```

© 2026 Information Processing Society of Japan

```
    "k_finalized ...  st bf k",
    "justified ...  st bj h2", "bj ≠ bf" ]
}
```

3.2 must-establish 分解と仕様不明示前提の二値測定

Lean 4 の定理は，それが成立するための前提条件を型シグネチャ上に明示的に持つ．定理の hypotheses には，型パラメータ・型クラスインスタンス・モデルパラメータ（バリデータ集合，ステーク関数など Stage 1 のモデル化に属する部品）と，実装が実際に確立せねばならない実体的条件が混在する．分解では前者を `depend-allowed`，後者を `must-establish` として分類し，`must-establish` 条件のみをプロパティ化する．平易には，must-establish は定理の成立に実体的に必要な条件，depend-allowed は型・モデルの都合による部品である．なお must-establish には，実装が手続き的に確立すべき条件と，定理が依拠する脅威モデル上の仮定（クォーラム交差が咎められないことなど）の双方が含まれる．8 ファイル・17 定理から，この手続きにより 27 の CHK-* 監査項目が導かれ，合計 57 の must-establish 前提条件が特定された．27 項目のうち 17 項目が非空の must-establish 条件を持ち，10 項目は持たない（後者は主に slashing 条件と executable 層の補題で，条件が仕様に直接定義されるため追加の前提を要さない）．この内訳は付録 A から検算できる．

二値ラベリング（E: 明示／非 E）． 導出された各前提条件について，consensus-specs（commit `2359a5e`）との突き合わせを行い，二値でラベリングする．**E**（Explicit）は「仕様書の散文に，安全性要件として等価な要件記述が存在する」，**非 E** は「散文に安全性要件としての等価な記述が見当たらない（実装コードとしてのみ存在する場合，および完全に不在の場合を含む）」と定義する．判定はキーワード照合に基づく機械判定器，および独立した 6 個の LLM モデルによる計 7 アノテータで付与する（いずれも自動判定であり，人手のアノテータは含まない；6 章）．主指標は非 E/n，すなわち仕様書の散文に安全性要件として明示されていない前提条件の割合とする．なお参考として併記する三値ラベリングは，**E**（明示）／**I**（Implicit: 実装コードや状態としては存在するが安全性要件としては述べられない）／**A**（Absent: 仕様に対応物が見当たらない）と定義し，二値では I と A をまとめて非 E とする．三値はアノテータ間で再現しないため（6 章），本稿の主張は二値に限る．

3.3 交差分析

CHK-* 監査項目は，Lean 定理（前提条件）と ethereum-vuln-dataset（過去のクライアント実バグデータベース）の交差から導かれる．27 項目の severity 分布は CRITICAL 13 / HIGH 11 / MEDIUM 3 である．このうち 14 項目（51.9%）が実バグ evidence を持つが，この 14 は 14 種類の別個の事故ではなく，3 クライアント（Nimbus / Lighthouse / Lodestar）の実質 6 件のユニークな実バグに対応する（複数の CHK-* が同一バグを異なる定理経由で参照するため，項目数と一意バグ数は一致しない；付録 A）．実バグとの接続を示せる経路はこの交差分析のみであり，示すのは label レベルの位置的共起にとどまり因果は主張しない（開示は 4.2 節）．

表 1 二値ラベリング結果（$n = 57$ 前提条件，7 アノテータ）
Table 1 Binary labeling results ($n = 57$, 7 annotators).

指標	値
全 7 アノテータが全会一致で非 E	**48/57（84.2%）**
機械判定器による点推定（非 E）	51/57（89.5%）
アノテータ間の非 E 割合の範囲	84.2%–100%
全アノテータが E と判定した条件	**0 件**
平均ペア一致率（21 ペア，$n = 57$）	0.913
Fleiss κ（$n = 57$）	0.394
PABAK（$n = 57$）	0.826

4. 分析結果

4.1 二値ラベリングの結果

27 の CHK-* 項目のうち非空の `lean_must_establish` を持つのは 17 項目であり（残り 10 項目は空），これら 17 項目（母定理は distinct 11 定理）から合計 57 の must-establish 前提条件が導出された（内訳検算は付録 A の `me` 列および付録 B の 表 A·3）．各前提条件を，仕様書に安全性要件として明示されている（E）か明示されていない（非 E）かの二値で，7 アノテータ（機械判定器＋独立 6 モデル）により判定した．

主指標． 57 前提条件のうち 48 件（84.2%）が 7 アノテータ全会一致で「安全性要件として明示されていない（非 E）」と判定された（**表 1**）．機械判定器単独による点推定は 51/57（89.5%），アノテータ間の非 E 割合の範囲は 84.2%–100% である．全アノテータが E と判定した条件は 0 件であった．すなわち，どの前提条件についても「仕様書に安全性要件として明示されている」と全員が同意する事例は存在しない．

信頼性． 条件単位（$n = 57$）で 21 ペアの平均一致率は 0.913，Fleiss $\kappa = 0.394$，PABAK $= 0.826$ である（概念単位 $n = 10$ では 0.838・0.278・0.676）．κ が中程度である一方で一致率と PABAK が高いのは，E の基準率が約 8%（条件単位 7.8%）と低いことによる κ の縮小効果（prevalence effect）であり，二値の一致それ自体は高い．

概念レベルの構造． $n = 57$ は延べ件数であり，同一概念が複数の定理へ反復出現するため独立標本ではない（6 章）．57 条件に含まれる distinct な概念は 10 個にとどまり，このうち 7 概念（延べ 48 条件）が 7 アノテータ全会一致で非 E であった（概念別の

© 2026 Information Processing Society of Japan

集約は付録の **表 A·2**）．割れた 3 概念（justified・finalized・votes_from_target_vset_property）は，いずれも仕様書が state.current_justified_checkpoint / state.finalized_checkpoint などの明示的な状態として扱う対象であり，「明示された状態」と「安全性要件としての明示」の境界で解釈が分かれる．

全会一致で非 E となった 7 概念のうち，q_intersection_slashed・quorum_2・supermajority_link については，探索した範囲（付録 B）で仕様書に対応する状態表現も安全性要件の記述も確認できなかった．一方 k_finalized・not_ancestor は，state.finalized_checkpoint や祖先判定など対応する状態表現を仕様書に持つが，k 一般化された形および「祖先でないこと」を安全性の要件として述べる散文は見当たらない．finalized（延べ 1）が割れ，k_finalized（延べ 19）が全会一致で非 E となるのはこの差による．非 E の 7 概念のうち height_cmp・inequality は，高さの比較や不等式といった証明構造に由来する補助条件であり，プロトコルの安全性そのものを担う条件ではない（この切り分けは Lean 定義に基づく）．残る q_intersection_slashed・quorum_2・k_finalized・supermajority_link・not_ancestor の 5 概念（延べ 42 条件）が，quorum 交差と正当化のアカウンタビリティという安全性の実体に対応する．なお延べ件数の内訳は概念間で偏っており，k_finalized（19 条件）と not_ancestor（16 条件）で全体の 61% を占める．概念単位で数えた場合，全会一致で非 E となるのは 10 概念中 7 概念（70%）である．

探索は consensus-specs 全 89 ファイルを対象とする（詳細は付録 B）．稼働中フォークに quorum の語は存在しない一方で，安全性の**帰結**を非形式に述べる散文は存在する．weak-subjectivity.md は「相反するチェックポイントが finalize されれば攻撃者は最低閾値（1/3）スラッシュされる」という帰結を，fast-confirmation.md は Byzantine 割合の閾値をそれぞれ散文で言及している．したがって本稿の非 E 判定が意味するのは「安全性の概念が仕様に一切現れない」ことではなく，「安全性定理が要求する**分解された前提**が，実装・監査が確認できる**検査可能な安全性要件**としては与えられていない」ことである（測定の限界は 6 章）．

4.2 主事例（動機付けの一例）

CHK-GEN-09（定理 no_k_finalized_justified_same_height，severity CRITICAL）の Lean 前提条件 ¬q_intersection_slashed（クォーラム交差が未スラッシュであること，honest-majority 仮定に対応）を取り上げる．この前提の Lean 定義は実在し（gasper-lean4, Core/AtomicDef/Quorums.lean:146），定理はこの前提のもとに，同一高さで相反する 2 つの正当化・ファイナリゼーションが同時に成立しないことを保証する．この非競合性が破れれば chain split に至る．

仕様探索（非 E）． この前提条件について，探索プロトコル（シンボル名 grep → 同義語 grep → 関連ファイル通読）に従い consensus-specs を全ファイル検索した．q_intersection_slashed/ quorum intersection / honest-majority / 競合 finalization を**検査可能な安全前提**とする散文記述は稼働中フォークに出現しなかった（quorum の語自体が出現しない）．process_justification_and_finalization 周辺は正当化・ファイナリゼーションの 2/3 算術（previous_epoch_target_balance * 3 >= total_active_balance * 2）を記述するのみで，クォーラム交差が未スラッシュであるという前提を明示しない．7 アノテータ全会一致で本前提を非 E と判定した（割れなし．表 A·3 の #39）．

同一 label 圏に対応づく実バグ． ethereum-vuln-dataset との交差分析により，CHK-GEN-09 は 2 件の実バグ evidence を持つ（付録 A）．**Nimbus PR#461**（Fix an overflow in the justification bits processing, root_cause=integer_overflow_underflow, impact_type=chain_split）および **Lighthouse PR#4576** である．PR#461 は正当化ビット処理の整数オーバーフローを修正したものであり，データセット上 impact_type=chain_split に分類されている．この前提条件と両バグは，いずれも label beacon-chain:justification-and-finality を共有する．

この対応の性質． (a) 共起の根拠は label の一致であって，両者が同一のコードパスを通るという主張ではない（コードパス同一性は本稿では確認していない）．(b) CHK-* 自体が ethereum-vuln-dataset の実バグの分布を参照して合成されているため，evidence 対応は「独立に観測された一致」ではなく「合成時に参照した分布への写像」でもある．以上より，本稿は因果を主張しない．示すのは，前提条件 ¬q_intersection_slashed が探索範囲で仕様に文章化されておらず（全会一致で非 E），かつ同一 label 圏で過去に chain split（nimbus#461）が実際に修正された，という位置的共起に限られる．これは「仕様準拠 = 安全」の反証ではなく，仕様に無標の安全前提が安全性クリティカルな処理を伴いうるという問題設定を支える動機付けの一例である．

4.3 コントロールサンプル

二値ラベリング手続きが E を返しうることを，期待ラベルが自明なサンプルで確認する．Slashing 条件（double / surround vote）は，is_slashable_attestation_data に

© 2026 Information Processing Society of Japan

`# Double vote` / `# Surround vote` という形で条件そのものが名指しされており，期待ラベル E に対し実際も E と判定された．ここで一点，判定基準の境界を明示しておく．4.1 節の非 E の定義は「実装コードとしてのみ存在する場合」を非 E に含めるが，本サンプルで E の根拠となったのは仕様中のコードに付された条件の名指しである．したがって本コントロールが確認するのは，判定手続きが条件を名指しする記述を E と識別できるという**手続き上の健全性**に限られ，E/非 E の境界そのものの妥当性を確認するものではない．非 E 側の割合の正しさは複数アノテータの一致で担保する．また本稿は「repo 全体で不在」方向のコントロールを論証に用いない．二値の非 E は，`specs/**/*.md` 全 89 ファイルの網羅探索の範囲で，安全性要件としての明示の有無のみを問う主張である．

4.4 CHK-* による findings トリアージの設計

本稿は下流の監査（Phase 02c 以降）を扱わない（6 章）．ここでは前半の出力である CHK-* が下流の findings トリアージにおいて持つ設計上の役割のみを述べる．CHK-* は 27 項目・57 前提条件という**定理レベル**の少数のアンカーである．一方，クライアント監査が生む findings は**コードレベル**の大量の指摘であり，両者は同一平面上の集合ではない．定理レベルのアンカーは「どの findings が安全性定理の前提条件に接続するか」を判定する参照座標として機能しうる．本稿の二値分析から言えるのは，48/57（84.2%）の前提条件が仕様書に安全性要件として明示されていないという**仕様側の事実**である．これが監査上の実害ある盲点であるかは未検証であり（6 章），本稿は判断を留保する．また「CHK-* を入力に据えれば，監査が生む findings のうち安全性定理の前提条件に接続するものの割合が実際に改善する」という下流の効果も本稿の対象外である．したがって本稿は，CHK-* がトリアージの参照座標となりうるという**設計上の位置づけの提示**にとどめ，基準の具体化と評価は今後の課題とする．

5. 関連研究

LLM による証明生成（逆方向・補完的）． IsabeLLM [3], [4] は，LLM を用いて自然言語仕様やコードから Isabelle/HOL の形式証明を自動生成する．これは「コード／仕様 → 証明」という方向であり，本稿の「人手で確立した証明 → 実装が満たすべき前提条件」という方向とは逆かつ補完的である．IsabeLLM は証明を組み立てる過程での LLM のハルシネーションを証明支援系の型検査で排除するが，そもそも何を定理とし何を前提とすべきかは人間が与える．本稿は，この「何を前提とすべきか」を Lean 証明の hypotheses から機械的に取り出し，仕様書との差分として可視化する．

LLM 監査の実世界ギャップとトリアージ． ZeroFalse [5] は静的解析の偽陽性を LLM で削減する手法だが，制御環境での F1 = 0.910 が実世界データセットでは 0.372 まで低下することを報告している（ただし同論文の主結果は CWE 特化の設定であり，別設定では F1 = 0.955 に達する）．この乖離は，LLM 監査の本質的課題が「検出」よりも「大量の findings から真に安全性に関わるものを選別する基準」にあることを示す．Refute-or-Promote [6] は生成された指摘を反証または昇格させて精度を担保する．これらが findings のテキスト内容に基づく後処理でトリアージするのに対し，本稿は findings の来歴 — 形式証明の前提条件に由来するか否か — という直交する信号を与える．証明済み定理へ辿れる（descends-from-proved）プロパティは，その severity が仕様書の記述量ではなく安全性定理の構造から与えられるため，仕様書に明示されない（非 E）前提であっても優先度が落ちない．

形式検証をめぐる実務言説． Buterin の形式検証概観 [7]（2026）は，Ethereum エコシステムで形式検証への関心が高まっていることを示すが，その焦点は「実装コードを証明によって正しいと示す」ことにある．本稿が扱うのはその一段手前の問い — 「何を証明すべきか，そしてその証明が要求する前提は仕様書のどこに書かれているか」である．証明対象のコードが仕様に完全準拠していても，証明が要求する前提条件が仕様に安全性要件として明示されていなければ，準拠は安全を含意しない．

自己引用との差分． 先行研究 [1], [2] との差分は，プロパティの来歴を人手で確立した Lean 証明に固定した点（LLM-only → Lean-anchored）にある．前提条件の抽出は LLM を介さない機械変換であるため，各プロパティが証明の仮説に対応することが構成上保証される．

6. おわりに

本稿では，人手で確立した Casper FFG の安全性証明（`gasper-lean4`．分析対象は 8 ファイル 17 定理，うち核心の安全性定理は 5 件）から前提条件を機械的に分解し，Ethereum コンセンサス仕様書に安全性要件として明示されるか否かの二値ラベリングによって，仕様に明示されない安全性前提の分布を定量化した．主指標（$n = 57$）は，57 前提条件のうち 48 件（84.2%）が 7 アノテータ全会一致で「安全性要件として明示されていない（非 E）」と判定され，全アノテータが E と判定した条件は 0 件，稼働中フォーク（phase0 から electra）では `quorum` の語が出現しない（gloas を除く），というものである．前提抽出のパイプラインは LLM を介さず，形式証明は専門家が 1 回だけ書けばよく，監査を実行する現場は Lean を一切読まずに JSON プロパティを受け取るだけでよい．

限界

本稿の測定には次の限界がある．

© 2026 Information Processing Society of Japan

(1) **アノテータはいずれも非人手である**: 二値判定は機械判定器と独立 6 モデルの LLM から成る 7 アノテータによる．二値の一致は高い（$n = 57$ で一致率 0.913・PABAK 0.826）が，正式な人手アノテーションは今後の課題である．

(2) **利益相反（COI）**：機械判定器の設計者は本稿の著者であり，結論が成立する方向へ数字が寄りうる利益相反を否定できない．独立 6 モデルの併用はこれを部分的に緩和するにとどまる．

(3) **二値ラベリングはヒューリスティックである**: Stage 2 の前提抽出は決定的で LLM 非介在だが，抽出後の二値ラベリングは機械ヒューリスティックであり，誤ラベルを生じうる．

(4) **三値（E/I/A）は再現困難である**: 三値内訳は 6 独立モデルで A の割合が 10.5%–78.9% に振れ，機械判定器はいずれのモデルとも一致しなかった．よって三値は主指標とせず，三値ラベリングの再現困難性そのものを付録 B の知見として報告する．

(5) **探索の範囲と精密化**:「非 E」判定の探索は consensus-specs 全 89 ファイルに対して完了した．その結果，`quorum` 語の非出現（稼働中フォーク）は確認したが，同時に安全性の**帰結**を非形式に述べる散文（`weak-subjectivity.md`）と Byzantine 閾値の言及（`fast-confirmation.md`）が存在することも判明した．したがって本稿の非 E は「安全性概念の完全な不在」ではなく「**分解された前提が検査可能な安全性要件として与えられていない**」ことを意味する（付録 B）．

(6) **標本は小さく独立でない**: $n = 57$ は延べ件数であり，同一概念が複数の定理へ反復出現するため独立標本ではない（distinct 概念は 10）．分析は Casper FFG の justification / finalization という単一構成要素に限られる．

(7) **下流の監査は未評価である**: 本稿はスコープを前半（プロパティ生成）の分析に限定し，下流の監査（`02c` 以降）は対象としない．その実効性の検証は今後の展望に述べる．

今後の展望

本稿の測定は前半（前提抽出とラベリング）に限られ，その含意 — 仕様のみを読む監査の盲点 — は論証にとどまる．ここで争点を明確にしておく．honest-majority のような脅威モデルの大域的仮定を，仕様が実装向けの要件として書くべきだとは主張しない．問うのは，実装が手続き的に確立すべき前提（投票の二重性検査，surround 判定，正当化ビットの更新規則など，図 1 の機構を実装側で担保する条件）が，仕様テキストのみを入力とする監査で拾えるか否かである．以下の検証がこの含意を実測へ進める．

(1) 下流監査の実効性検証（最優先）． 本稿がスコープ外とした Stage 3 を小規模に実行する．ある実バグの修正前コミット（例：付録 A の nimbus#461，justification-bits オーバーフロー，`impact_type=chain_split`）を対象に，(A) Lean 由来の must-establish プロパティと (B) 仕様テキストのみから生成した LLM プロパティを監査入力として与え，当該 chain split の根因への到達・偽陽性・前提被覆率を比較する．これにより，84.2% の差分が単に仕様の役割の産物か，監査上実害のある盲点かを判別できる．Lighthouse・Lodestar への拡張で汎用性も測る．

(2) 他 BFT プロトコルへの横展開． 同手続きを Tendermint・HotStuff 等の BFT 系ファイナリティに適用し，「安全性の帰結は散文に述べられるが，分解された前提は無標」というパターンが Ethereum 固有か BFT 系に共通かを調べる．共通であれば，安全性定理が要求する前提が仕様に検査可能な要件として明示されているか，という本手法の問いの一般性を裏づける．

参考文献

[1] Kamba, M. and Sannai, A.: SPECA: Specification-to-Checklist Agentic Auditing for Multi-Implementation Systems — A Case Study on Ethereum Clients, arXiv:2602.07513 (2026).

[2] Kamba, M., Murakami, H. and Sannai, A.: Beyond Code Reasoning: Specification-Anchored Auditing of Multi-Implementation Distributed Protocols, arXiv:2604.26495 (2026).

[3] IsabeLLM: LLM-Guided Proof Synthesis for Interactive Theorem Provers, arXiv:2601.07654 (2026).

[4] IsabeLLM v2, arXiv:2606.18098 (2026).

[5] ZeroFalse: Reducing False Positives in LLM-Based Vulnerability Detection, arXiv:2510.02534.

[6] Refute-or-Promote: Precision-Preserving Triage of LLM-Generated Findings, arXiv:2604.19049 (2026).

[7] Buterin, V.: A shallow dive into formal verification, `https://vitalik.eth.limo/` (2026).

[8] Buterin, V. and Griffith, V.: Casper the Friendly Finality Gadget, arXiv:1710.09437 (2017).

[9] Buterin, V., Hernandez, D., Kamphefner, T. et al.: Combining GHOST and Casper, arXiv:2003.03052 (2020).

[10] NyxFoundation: gasper-lean4 — A Lean 4 formalization of the Casper FFG finality gadget (commit `7650709`).

[11] Ethereum: Consensus Specifications (commit `2359a5e`).

[12] SPECA Project: speca-lean4-plugin — Deterministic must-establish extraction from Lean 4 proofs (tag `20260723-gasper`).

[13] NyxFoundation: ethereum-vuln-dataset — A curated dataset of historical Ethereum client vulnerabilities (commit `c7c4ccb`).

付　　録

A.1 CHK-* 全 27 項目一覧

各 CHK-* は `gasper-lean4` の 1 定理に由来し，対応する consensus-specs のセクションと，過去のクライアント実バグ

© 2026 Information Processing Society of Japan

表 A·1　CHK-* 27 項目（severity・母定理・me・実バグ evidence）
Table A·1　The 27 CHK-* items.

ID	sev	me	vuln evidence
CHK-AS-01/02/03	C	4	—
CHK-GEN-01/02	C	4	—
CHK-GEN-03	H	0	—
CHK-GEN-04	H	2	—
CHK-GEN-05	C	4	—
CHK-GEN-06	H	0	nimbus#461, lh#4576
CHK-GEN-07	C	4	nimbus#461, lh#4576
CHK-GEN-08	C	6	nimbus#461, lh#4576
CHK-GEN-09	**C**	**4**	**nimbus#461, lh#4576**
CHK-GEN-10	C	4	nimbus#461, lh#4576
CHK-GEN-11	C	3	lh#5037, nimbus#2392
CHK-GEN-12	H	0	nimbus#461, lh#4576
CHK-JF-01	H	0	nimbus#461, lh#4576
CHK-JF-02	M	1	—
CHK-JF-03	C	1	nimbus#461, lh#4576
CHK-JF-04	M	0	—
CHK-LV-01	M	2	—
CHK-QW-01	H	4	lh#9106, lodestar#3045
CHK-QW-02	H	2	—
CHK-QW-03	H	0	—
CHK-SL-01	C	0	lh#5037, nimbus#2392
CHK-SL-02/03/04	H	0	lh#5037, nimbus#2392

（ethereum-vuln-dataset [13]）への位置的対応を持つ．sev 列は severity（C: CRITICAL, H: HIGH, M: MEDIUM），me 列は**1 項目あたりの** must-establish 前提条件数である．ID 欄が CHK-AS-01/02/03 のように複数項目を 1 行にまとめた行では，同じ値が各項目に成り立つ（例：CHK-AS-01/02/03 は各 4 件で計 12 件）．したがって列の総和は $4{\times}3{+}4{\times}2{+}2{+}4{+}4{+}6{+}4{+}4{+}3{+}1{+}1{+}2{+}4{+}2 = 57$ となる．各項目が参照する consensus-specs のセクションは beacon-chain.md・fork-choice.md・p2p-interface.md の 3 ファイルにまたがっており（紙幅の都合により列は省略した），探索が単一ファイルに閉じていないことの傍証である．

合計 27 項目（CRITICAL 13 / HIGH 11 / MEDIUM 3），must-establish 前提条件の総数は 57，非空（$me \geq 1$）17 項目・空 10 項目である（**表 A·1**）．evidence あり 14 項目（51.9%）は 14 種類の独立事故ではなく，3 クライアントの実バグ 6 件（nimbus#461, lh#4576, lh#5037, nimbus#2392, lh#9106, lodestar#3045）へ集約される．この 6 件は ethereum-vuln-dataset（commit c7c4ccb）の該当エントリと突き合わせ，root_cause・impact_type・共有 label を確認した（4 件が impact_type=chain_split）．本表の vuln evidence 列は当該 CHK-* と実バグが同一 label を共有する**位置的共起**を示すものであり，コードパスの同一性や因果を示すものではない（4.2 節の開示を参照）．

表 A·2　57 条件の概念別集約（延べ件数・三値・二値）
Table A·2　The 57 preconditions aggregated by concept.

概念	延べ	三値	二値（7 アノテータ）
k_finalized	19	I	非 E（7/7）
not_ancestor	16	I	非 E（7/7）
justified	6	E	割れ（E 4/7）
q_intersection_slashed	4	A	非 E（7/7）
height_cmp	3	I	非 E（7/7）
inequality	3	I	非 E（7/7）
quorum_2	2	A	非 E（7/7）
votes_from_target_vset_property	2	I	割れ（E 2/7）
supermajority_link	1	I	非 E（7/7）
finalized	1	I	割れ（E 3/7）
計	57	—	非 E 48・割れ 9

A.2　must-establish 57 条件の概念別内訳と探索

主指標は二値（E ／非 E）である．表 A·2 に 57 条件の概念別集約を，表 A·3 に 57 条件の全項目を示す．三値の列は機械判定器を 1 回だけ実行して得た参考値である．

主事例 CHK-GEN-09 の前提 ¬q_intersection_slashed は表 A·3 の #39 であり，三値では A，二値では 7 アノテータ全会一致で非 E である．q_intersection_slashed の Lean 定義は実在する（Core/AtomicDef/Quorums.lean:146）．slashable_bound の 4 条件（k_finalized×2, not_ancestor×2）は gasper-lean4（commit 7650709）の定義に一致することを実物照合で確認済みである．

三値ラベリングの非再現性．　参考として機械判定器の三値集計は E = 6（10.5%, 95% CI [5, 21]）／ I = 45（78.9%）／ A = 6（10.5%）である（$n = 57$, I + A = 51 = 89.5%）．三値 E/I/A は独立 6 モデルの間で A/n が 10.5%–78.9% に広く振れ，モデル間 Fleiss $\kappa = 0.381$（三値），機械判定器の点推定は他アノテータのいずれとも一致しなかった．よって三値は主指標とせず，本稿の主張は二値に限定する．

探索の網羅性（全ファイル探索の結果）．　二値の非 E 判定に用いた探索は，consensus-specs（commit 2359a5e）の specs/**/*.md 全 89 ファイル（稼働中フォーク 50・次期／実験 39）に対して完了した．探索プロトコルは (1) シンボル名 grep，(2) 同義語・言い換え grep（honest majority / quorum intersection / safety assumption / conflicting finalization 等），(3) 関連ファイル通読の 3 段からなる．結果，(i) quorum の語は稼働中フォーク（phase0 から electra）に 0 ヒットであり，唯一の出現は次期フォーク gloas の get_builder_payment_quorum_threshold（ビルダー支払い閾値，FFG の安全性 quorum とは無関係）であった．一方，(ii) weak-subjectivity.md は安全性の**帰結**（相反チェックポイントの finalize ⇒ 最低 1/3 スラッシュ）を非形式に散文で述べ，(iii) fast-confirmation.md

© 2026 Information Processing Society of Japan

表 A·3　must-establish 57 条件の一覧（me 列の内訳，二値判定つき）

Table A·3　The 57 must-establish preconditions.

#	CHK-*	must-establish 条件	概念	二値
1	CHK-AS-01	k_finalized ... st b1 k1	k_finalized	非 E 7/7
2	CHK-AS-01	k_finalized ... st b2 k2	k_finalized	非 E 7/7
3	CHK-AS-01	not_ancestor parent b1 b2	not_ancestor	非 E 7/7
4	CHK-AS-01	not_ancestor parent b2 b1	not_ancestor	非 E 7/7
5	CHK-AS-02	k_finalized ... st b1 k1	k_finalized	非 E 7/7
6	CHK-AS-02	k_finalized ... st b2 k2	k_finalized	非 E 7/7
7	CHK-AS-02	not_ancestor parent b1 b2	not_ancestor	非 E 7/7
8	CHK-AS-02	not_ancestor parent b2 b1	not_ancestor	非 E 7/7
9	CHK-AS-03	k_finalized ... st b1 k1	k_finalized	非 E 7/7
10	CHK-AS-03	k_finalized ... st b2 k2	k_finalized	非 E 7/7
11	CHK-AS-03	not_ancestor parent b1 b2	not_ancestor	非 E 7/7
12	CHK-AS-03	not_ancestor parent b2 b1	not_ancestor	非 E 7/7
13	CHK-GEN-01	k_finalized ... st b1 k1	k_finalized	非 E 7/7
14	CHK-GEN-01	k_finalized ... st b2 k2	k_finalized	非 E 7/7
15	CHK-GEN-01	not_ancestor parent b1 b2	not_ancestor	非 E 7/7
16	CHK-GEN-01	not_ancestor parent b2 b1	not_ancestor	非 E 7/7
17	CHK-GEN-02	k_finalized ... st b1 k1	k_finalized	非 E 7/7
18	CHK-GEN-02	k_finalized ... st b2 k2	k_finalized	非 E 7/7
19	CHK-GEN-02	not_ancestor parent b1 b2	not_ancestor	非 E 7/7
20	CHK-GEN-02	not_ancestor parent b2 b1	not_ancestor	非 E 7/7
21	CHK-GEN-04	¬ q_intersection_slashed ... st	q_intersection_slashed	非 E 7/7
22	CHK-GEN-04	votes_from_target_vset_property vset st	votes_from_target_vset_property	割れ E2/7
23	CHK-GEN-05	k_finalized ... st b1 k1	k_finalized	非 E 7/7
24	CHK-GEN-05	k_finalized ... st b2 k2	k_finalized	非 E 7/7
25	CHK-GEN-05	not_ancestor parent b1 b2	not_ancestor	非 E 7/7
26	CHK-GEN-05	not_ancestor parent b2 b1	not_ancestor	非 E 7/7
27	CHK-GEN-07	k_finalized ... st b1 k1	k_finalized	非 E 7/7
28	CHK-GEN-07	k_finalized ... st b2 k2	k_finalized	非 E 7/7
29	CHK-GEN-07	h2 < h1	height_cmp	非 E 7/7
30	CHK-GEN-07	not_ancestor parent b2 b1	not_ancestor	非 E 7/7
31	CHK-GEN-08	justified ... st s s_h	justified	割れ E4/7
32	CHK-GEN-08	supermajority_link ... st s t s_h t_h	supermajority_link	非 E 7/7
33	CHK-GEN-08	k_finalized ... st fin...	k_finalized	非 E 7/7
34	CHK-GEN-08	final_h < t_h	height_cmp	非 E 7/7
35	CHK-GEN-08	s_h < final_h	height_cmp	非 E 7/7
36	CHK-GEN-08	not_ancestor parent final t	not_ancestor	非 E 7/7
37	CHK-GEN-09	k_finalized ... st bf k	k_finalized	非 E 7/7
38	CHK-GEN-09	justified ... st bj h2	justified	割れ E4/7
39	**CHK-GEN-09**	¬ q_intersection_slashed ... st	q_intersection_slashed	**非 E 7/7**
40	CHK-GEN-09	bj ≠ bf	inequality	非 E 7/7
41	CHK-GEN-10	justified ... st b1 h1	justified	割れ E4/7
42	CHK-GEN-10	justified ... st b2 h2	justified	割れ E4/7
43	CHK-GEN-10	¬ q_intersection_slashed ... st	q_intersection_slashed	非 E 7/7
44	CHK-GEN-10	b1 ≠ b2	inequality	非 E 7/7
45	CHK-GEN-11	justified ... st b1 h	justified	割れ E4/7
46	CHK-GEN-11	justified ... st b2 h	justified	割れ E4/7
47	CHK-GEN-11	b1 ≠ b2	inequality	非 E 7/7
48	CHK-JF-02	finalized ... st p h	finalized	割れ E3/7
49	CHK-JF-03	k_finalized ... st b k	k_finalized	非 E 7/7
50	CHK-LV-01	¬ q_intersection_slashed ... st	q_intersection_slashed	非 E 7/7
51	CHK-LV-01	votes_from_target_vset_property vset st	votes_from_target_vset_property	割れ E2/7
52	CHK-QW-01	k_finalized ... st bL kL	k_finalized	非 E 7/7
53	CHK-QW-01	k_finalized ... st bR kR	k_finalized	非 E 7/7
54	CHK-QW-01	¬ hash_ancestor parent bL bR	not_ancestor	非 E 7/7
55	CHK-QW-01	¬ hash_ancestor parent bR bL	not_ancestor	非 E 7/7
56	CHK-QW-02	quorum_2 ... qL vL	quorum_2	非 E 7/7
57	CHK-QW-02	quorum_2 ... qR vR	quorum_2	非 E 7/7

© 2026 Information Processing Society of Japan

表 A·4　CHK-* を導出した 17 定理（ファイル層別）
Table A·4　The 17 theorems that derive the CHK-* items.

層・ファイル	定理数	CHK-*
Core/Theories/AccountableSafety	5	10
Core/Theories/SlashableBound	3	3
Core/Theories/PlausibleLiveness	1	2
Core/Lemmas/AccountableSafety	2	2
Core/Lemmas/Justification	2	2
Executable/Justification	1	3
Executable/Slashing	2	4
Executable/Quorums	1	1
計（8 ファイル）	17	27

は `CONFIRMATION_BYZANTINE_THRESHOLD` により Byzantine 割合の脅威モデルを標識している．いずれも安全性定理が要求する**分解された前提**（quorum 交差が未スラッシュであるという機構を含む 5 概念）を，実装・監査が確認できる検査可能な要件としては与えていない．したがって本稿の非 E は「概念の完全な不在」ではなく「検査可能な前提としての無標」を意味する．なお，正式な人手アノテーション（現状は非人手アノテータのみ）は今後の課題である（6 章）．

A.3　gasper-lean4 定理一覧（8 ファイル 17 定理）

Stage 1 で用いた形式証明成果物 [10]（commit `7650709`）のうち，CHK-* を導出した 17 定理を示す（**表 A·4**）．17 定理はすべて実在し，証明ステータスはすべて proved（`sorry` ゼロ）．安全性の核心は `Core/Theories/AccountableSafety.lean` の 5 定理である．CHK-* の合計は 27，導出された must-establish 前提条件の合計は 57 である．

主事例の母定理は `no_k_finalized_justified_same_height`（Core/Theories/AccountableSafety，CHK-GEN-09，CRITICAL）であり，その前提条件 ¬`q_intersection_slashed`（表 A·3 の #39，クォーラム交差の帰責に関する仮定，7 アノテータ全会一致で非 E）を持つ．同定理が位置する `process_justification_and_finalization` の justification-bits 処理には，同一 label（`justification-and-finality`）を共有する nimbus#461（`impact_type=chain_split`）および lh#4576 が位置的に共起する．共起の根拠は label の一致であってコードパスの同一性ではなく，因果は主張しない．

© 2026 Information Processing Society of Japan